VIETNAM NATIONAL UNIVERSITY – HOCHIMINH CITY

INTERNATIONAL UNIVERSITY

SCHOOL OF ELECTRICAL ENGINEERING

DIGITAL SYSTEM DESIGN LABORATORY

LAB 1: IMPLEMENTATION OF BASIC COMBINATIONAL LOGIC CIRCUITS IN FPGA USING VERILOG

Submitted by

Đặng Chí Thành – EEEEIU23023

Date Submitted:	26/10/2026
Date Performed:	26/10/2026
Lab Section:	Lab 01
Course Instructor:	Dr. Vo Minh Thanh

GRADING GUIDELINE FOR LAB REPORT

Number	Content		Score	Comment
1	**Format (max 9%)**			
	- Font type	Yes No		
	- Font size	Yes No		
	- Lab title	Yes No		
	- Page number	Yes No		
	- Table of contents	Yes No		
	- Header/Footer	Yes No		
	- List of figures (if exists)	Yes No		
	- List of tables (if exists)	Yes No		
	- Lab report structure	Yes No		
2	**English Grammar and Spelling (max 6%)**			
	- Grammar	Yes No		
	- Spelling	Yes No		
3	**Data and Result Analysis (max 85%)**			
Total Score				

Signature:

Date:

Table of Contents

List of Figures I
List of Tables II
Nomenclature III
1 Theoretical Background 1
1.1 Basic Logic Gates 1
1.2 Half Adder and Full Adder 2
1.3 4-bit Ripple Carry Adder 3
1.4 Multiplexers 4
1.5 Verilog Modeling Styles 7
2 Experimental Procedure 8
2.1 Experiment 1: Basic Logic Gates 9
2.2 Experiment 2: Half Adder 11
2.3 Experiment 3: Full Adder 12
2.4 Experiment 4: 4-bit Ripple Carry Adder 14
2.5 Experiment 5: 2:1 Multiplexer 16
2.6 Experiment 6: 4:1 Multiplexer 18
2.7 Experiment 7: 16:1 Multiplexer from Five 4:1 Multiplexers 20
3 Experimental Results 22
3.1 Experiment 1 22
3.2 Experiment 2 24
3.3 Experiment 3 26
3.4 Experiment 4 28
3.5 Experiment 5 30
3.6 Experiment 6 32
3.7 Experiment 7 34
4 Discussion of Results 38
4.1 Comparison of the Modeling Styles 38
4.2 Errors Found in the Reference Code 39
4.3 Practical Issues and Sources of Error 40
4.4 Conclusion 40

List of Figures

Figure 1: *Symbols and truth tables of the basic logic gates (from the lab manual).* 2
Figure 2: *Half adder circuit (from the lab manual).* 3

Figure 3: *Full adder circuit (from the lab manual).* 3
Figure 4: *Block diagram of the 4-bit ripple carry adder (from the lab manual).* 4
Figure 5: *2:1 multiplexer circuit (from the lab manual).* 5
Figure 6: *4:1 multiplexer circuit (from the lab manual).* 5
Figure 7: *16:1 multiplexer built from five 4:1 multiplexers (from the lab manual).* 6
Figure 8: *FPGA implementation results of Experiment 1.* 23
Figure 9: *Simulation results of Experiment 1.* 24
Figure 10: *FPGA implementation results of Experiment 2.* 25
Figure 11: *Simulation results of Experiment 2.* 26
Figure 12: *FPGA implementation results of Experiment 3.* 27
Figure 13: *Simulation results of Experiment 3.* 28
Figure 14: *FPGA implementation results of Experiment 4.* 29
Figure 15: *Simulation results of Experiment 4.* 30
Figure 16: *FPGA implementation results of Experiment 5.* 31
Figure 17: *Simulation results of Experiment 5.* 32
Figure 18: *FPGA implementation results of Experiment 6.* 33
Figure 19: *Simulation results of Experiment 6.* 34
Figure 20: *FPGA implementation results of Experiment 7.* 36
Figure 21: *Simulation results of Experiment 7.* 37
Figure 22: *Flow Summary of the three 4:1 MUX models (Experiment 6).* 39

List of Tables

Table 1: *Basic logic gates and their Verilog descriptions.* 1
Table 2: *Truth table of the half adder.* 3
Table 3: *Truth table of the full adder.* 3
Table 4: *Representative rows of the truth table of the 4-bit ripple carry adder.* 4
Table 5: *Truth table of the 2:1 multiplexer.* 6
Table 6: *Truth table of the 4:1 multiplexer (s1 = MSB).* 7
Table 7: *Truth table of the 16:1 multiplexer.* 7
Table 8: *I/O assignment of Experiment 1.* 9
Table 9: *I/O assignment of Experiment 4.* 14
Table 10: *Input selected by the reference 4:1 MUX modules.* 18
Table 11: *I/O assignment of Experiment 7.* 20
Table 12: *Experiment 1 – expected outputs and observed results.* 22
Table 13: *Experiment 2 – expected outputs and observed results.* 25
Table 14: *Experiment 3 – expected outputs and observed results.* 27
Table 15: *Experiment 4 – expected outputs and observed results.* 29
Table 16: *Experiment 5 – expected outputs and observed results.* 31
Table 17: *Experiment 6 – expected outputs and observed results.* 33
Table 18: *Experiment 7 – expected outputs and observed results.* 35
Table 19: *Errors found in the reference code and their corrections.* 39

Nomenclature

FPGA:	Field-Programmable Gate Array
HDL:	Hardware Description Language
LE:	Logic Element of the Cyclone II FPGA
DF, BH, GL/STRU:	Dataflow, Behavioral and Gate-level (Structural) models
SW[n]:	Slide switch n of the DE2 board (up = logic 1)
KEY[n]:	Push-button n of the DE2 board (active-low)
LEDG[n], LEDR[n]:	Green / red LED n of the DE2 board (on = logic 1)
A, B, Cin:	Adder data inputs and carry input
S, Cout:	Adder sum output and carry output
i0–i3, D0–D15:	Multiplexer data inputs
sel, s0–s3:	Multiplexer select inputs
y:	Multiplexer output
$\oplus$, ·, +, $\bar{X}$:	XOR, AND, OR and NOT operations

1 Theoretical Background

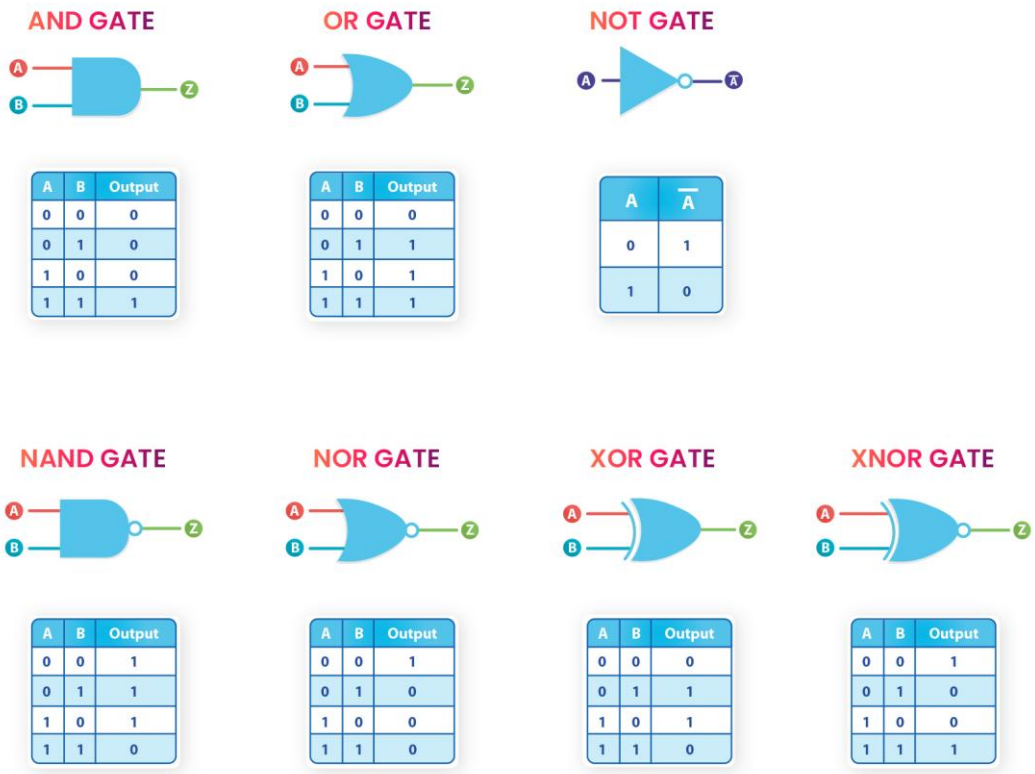
This laboratory implements seven basic combinational logic circuits in Verilog HDL and verifies them on an FPGA development board. A combinational circuit has no memory: its outputs depend only on the present values of its inputs, so each circuit can be fully described by a Boolean expression or a truth table. This section summarizes the theory of each circuit and the three Verilog modeling styles used in the experiments.

1.1 Basic Logic Gates

The seven basic logic gates and their Boolean expressions are listed in Table 1, together with the Verilog operator used in the dataflow model and the built-in primitive used in the gate-level model. Their symbols and truth tables are shown in Figure 1.

Table 1: *Basic logic gates and their Verilog descriptions.*

Gate	Boolean expression	Verilog operator	Verilog primitive
AND	$Y = A \cdot B$	`a & b`	`and`
OR	$Y = A + B$	`a | b`	`or`
NOT	$Y = \bar{A}$	`~a`	`not`
NAND	$Y = \overline{(A \cdot B)}$	`~(a & b)`	`nand`
NOR	$Y = \overline{(A + B)}$	`~(a | b)`	`nor`
XOR	$Y = A \oplus B$	`a ^ b`	`xor`
XNOR	$Y = \overline{(A \oplus B)}$	`~(a ^ b)`	`xnor`



A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

A	$\overline{A}$
0	1
1	0

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

Figure 1: *Symbols and truth tables of the basic logic gates (from the lab manual).*

1.2 Half Adder and Full Adder

A half adder adds two 1-bit inputs A and B. Because $1 + 1 = 10_2$, two outputs are required: the sum bit S and the carry bit Cout:

$$S = A \oplus B, \qquad Cout = A \cdot B$$

A full adder also accepts a carry input Cin from a less significant stage, so it adds three bits. The sum bit is HIGH when an odd number of inputs are HIGH, and the carry output is a majority function that is HIGH when at least two inputs are HIGH:

$$S = A \oplus B \oplus Cin, \qquad Cout = A \cdot B + B \cdot Cin + A \cdot Cin$$

The lab manual implements the full adder with two XOR gates, two AND gates and one OR gate (Figure 3), using the equivalent form $Cout = (A \oplus B) \cdot Cin + A \cdot B$. The truth tables of both adders are given in Table 2 and Table 3.

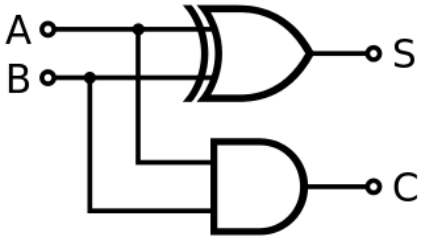


Figure 2: *Half adder circuit (from the lab manual).*

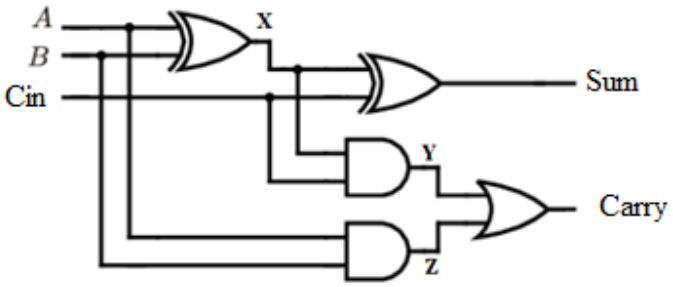


Figure 3: *Full adder circuit (from the lab manual).*

Table 2: *Truth table of the half adder.*

A	B	Cout	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Table 3: *Truth table of the full adder.*

A	B	Cin	Cout	S	Decimal sum
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	1	1
0	1	1	1	0	2
1	0	0	0	1	1
1	0	1	1	0	2
1	1	0	1	0	2
1	1	1	1	1	3

1.3 4-bit Ripple Carry Adder

A 4-bit ripple carry adder (RCA) is built by cascading four full adders (Figure 4). The carry output of each stage is connected to the carry input of the next, more significant stage, so the carry "ripples" from bit 0 to bit 3. The adder computes the 5-bit result {Cout, S3 S2 S1 S0} = A + B + Cin, where A and B are 4-bit unsigned numbers. Cout acts as the fifth

bit (weight 16) and is HIGH whenever the sum exceeds 15. The worst-case delay occurs when a carry generated in bit 0 must propagate through all four stages, e.g. $1111_2 + 0000_2 + 1$.

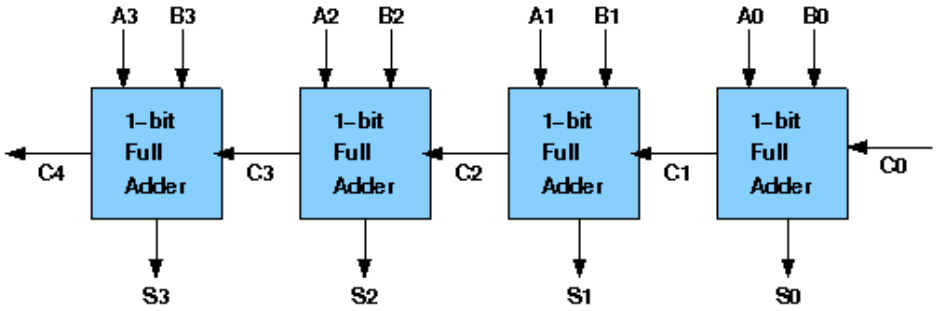


Figure 4: *Block diagram of the 4-bit ripple carry adder (from the lab manual).*

Because the adder has $2^9 = 512$ input combinations, Table 4 lists representative rows of its truth table (taken from the lab manual and rewritten with bit 3 as the most significant bit).

Table 4: *Representative rows of the truth table of the 4-bit ripple carry adder.*

A3A2A1A0	B3B2B1B0	Cin	Cout	S3S2S1S0	Decimal
0000	0000	0	0	0000	0 + 0 = 0
0100	0100	0	0	1000	4 + 4 = 8
1000	1000	0	1	0000	8 + 8 = 16
1010	1010	0	1	0100	10 + 10 = 20
1100	1100	0	1	1000	12 + 12 = 24
1110	1110	0	1	1100	14 + 14 = 28
1111	1111	0	1	1110	15 + 15 = 30
1111	0000	1	1	0000	15 + 0 + 1 = 16

1.4 Multiplexers

A multiplexer (MUX) routes one of several data inputs to a single output under the control of select inputs. A 2:1 MUX passes i0 when sel = 0 and i1 when sel = 1:

$$y = i0 \cdot \overline{sel} + i1 \cdot sel$$

A 4:1 MUX uses two select bits. In this report s1 is the most significant select bit, so s1 s0 = 00, 01, 10, 11 selects i0, i1, i2 and i3, respectively:

$$y = i0 \cdot \overline{s1} \cdot \overline{s0} + i1 \cdot \overline{s1} \cdot s0 + i2 \cdot s1 \cdot \overline{s0} + i3 \cdot s1 \cdot s0$$

A 16:1 MUX needs four select bits. It can be built as a two-level tree of five 4:1 multiplexers (Figure 7): four first-level MUXes share S1 S0 and each selects one input from a group of four (D0–D3, D4–D7, D8–D11, D12–D15); the second-level MUX uses S3 S2 to choose which group reaches the output. Hence y = D[S3 S2 S1 S0].

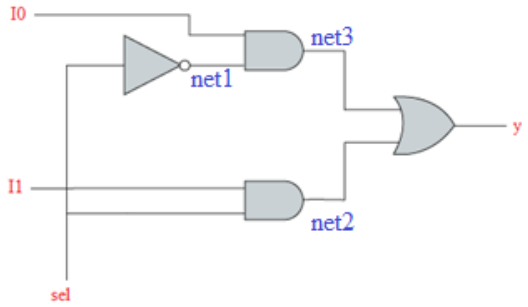


Figure 5: *2:1 multiplexer circuit (from the lab manual).*

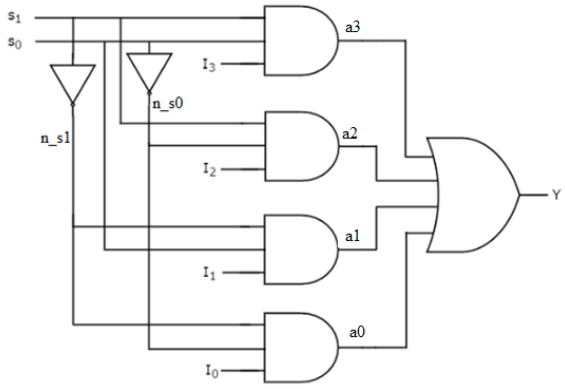


Figure 6: *4:1 multiplexer circuit (from the lab manual).*

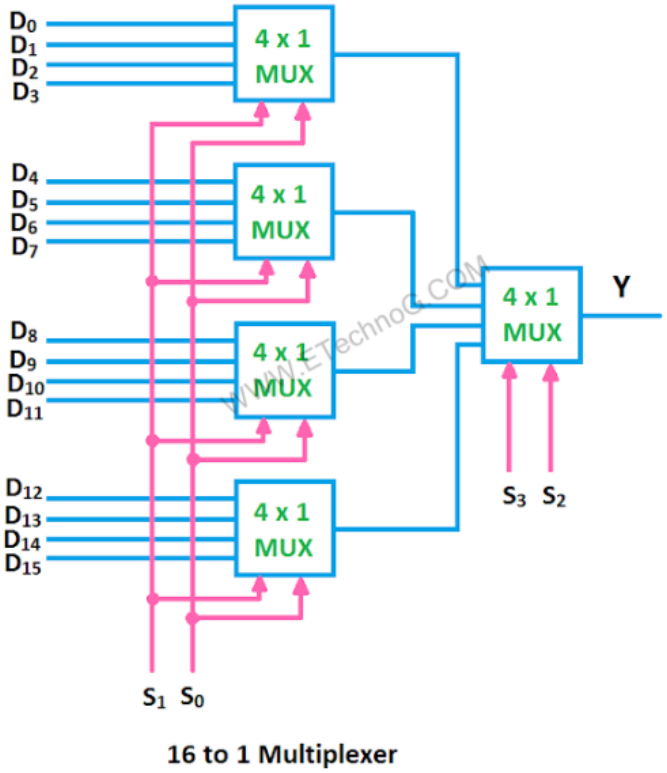


Figure 7: *16:1 multiplexer built from five 4:1 multiplexers (from the lab manual).*

The truth tables of the three multiplexers are given in Table 5, Table 6 and Table 7. For the 2:1 MUX the full 8-row table is shown; for the larger multiplexers the compact form is used, in which each row means that the output follows the listed data input regardless of the other data inputs.

Table 5: *Truth table of the 2:1 multiplexer.*

sel	i1	i0	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Table 6: *Truth table of the 4:1 multiplexer (s1 = MSB).*

s1	s0	y
0	0	i0
0	1	i1
1	0	i2
1	1	i3

Table 7: *Truth table of the 16:1 multiplexer.*

S3 S2 S1 S0	y	S3 S2 S1 S0	y
0000	D0	1000	D8
0001	D1	1001	D9
0010	D2	1010	D10
0011	D3	1011	D11
0100	D4	1100	D12
0101	D5	1101	D13
0110	D6	1110	D14
0111	D7	1111	D15

1.5 Verilog Modeling Styles

Each circuit is described in three styles, which differ only in the level of abstraction of the description:

- **Dataflow model** – continuous assignments (`assign`) that state the Boolean or arithmetic expression of each output.
- **Behavioral model** – procedural statements (`if`, `case`, `for`) inside an `always` block that describe how the outputs respond to the inputs. Outputs are declared as `reg`.
- **Gate-level (structural) model** – an explicit netlist of built-in primitives (`and`, `or`, `xor`, ...) or of lower-level module instances connected by `wire` signals.

Since all three descriptions represent the same Boolean functions, they are expected to produce identical outputs for every input combination, both in simulation and on the FPGA. The synthesis tool is also expected to map them to equivalent hardware.

2 Experimental Procedure

The following equipment and software were used:

- Altera DE2 development board with a Cyclone II EP2C35F672C6 FPGA, 18 slide switches (SW17–SW0), 4 push-buttons (KEY3–KEY0), 18 red LEDs (LEDR17–LEDR0) and 9 green LEDs (LEDG8–LEDG0).
- USB-Blaster cable (on-board) for JTAG programming and a 9 V DC power adapter.
- Quartus II 9.0 (Build 132) for design entry, synthesis, fitting and programming.
- ModelSim-Altera for functional simulation of the testbenches.

The lab manual refers to the DE2-115 board. Because a DE2 board (Cyclone II) was used, the Cyclone II pin assignment file `DE2_pin_assignments_Cyclone_II.csv` was imported instead of the DE2-115 file. The port names (SW, KEY, LEDG, LEDR) are identical on both boards, so the Verilog code does not depend on the board.

Every model was implemented as a separate Quartus II project using the following procedure:

1. Create a new project (File → New Project Wizard) and select the device family Cyclone II, device EP2C35F672C6.
2. Add the circuit module file and a top-level module file that connects the circuit ports to SW, KEY and LEDG; set the top-level module as the top-level entity.
3. Import the pin assignments (Assignments → Import Assignments) and set the unused pins to “As input tri-stated”.
4. Compile the design (Processing → Start Compilation) and check that the compilation is successful with 0 errors.
5. Program the FPGA through the USB-Blaster in JTAG mode (Tools → Programmer) with the generated .sof file.
6. Apply the test input combinations with the switches and compare the LED outputs with the truth table.

In every top-level module the red LEDs mirror the switches (`assign LEDR = SW;`) so that the applied input combination is visible on the board. Each model was also simulated

in ModelSim with a testbench that applies the input combinations and compares the outputs with the expected values.

Several errors were found in the reference code of the lab manual. They were corrected with minimal changes, which are marked with `// [FIX]` comments in the listings below and are summarized in Section 4.

2.1 Experiment 1: Basic Logic Gates

The goal of this experiment is to implement all seven basic logic gates with two inputs a and b. Table 8 shows the input/output assignment on the DE2 board.

Table 8: *I/O assignment of Experiment 1.*

Signal	Board I/O	Signal	Board I/O
a	SW[1]	AND	LEDG[5]
b	SW[2]	OR	LEDG[4]
		NOT a	LEDG[3]
		NAND	LEDG[2]
		NOR	LEDG[1]
		XOR	LEDG[0]
		XNOR	LEDG[6]

Dataflow model (allgate_DF.v):

```
module allgate_DF ( a, b, yand,yor,ynot,ynand,ynor,yxor,yxnor );
input a,b;
output yand, yor, ynot, ynand, ynor, yxor, yxnor;
assign yand = a & b;      // AND Operation
assign yor = a | b;       // OR Operation
assign ynot = ~a ;        // NOT Operation
assign ynand = ~(a & b);  // NAND Operation
assign ynor = ~(a | b);   // NOR Operation
assign yxor = a ^ b;      // XOR Operation
assign yxnor =~(a^b);     // XNOR Operation
endmodule
```

Behavioral model (allgate_BH.v):

```
// [FIX] The keyword 'assign' was removed inside the always block
//       (procedural continuous assignment is not synthesizable in Quartus II)
module allgate_BH ( a, b, yand,yor,ynot,ynand,ynor,yxor,yxnor );
input a,b;
output yand, yor, ynot, ynand, ynor, yxor, yxnor;
reg yand, yor, ynot, ynand, ynor, yxor, yxnor;
always @(*)
begin
    yand = a & b;      // AND Operation
    yor = a | b;       // OR Operation
    ynot = ~a ;        // NOT Operation
```

```
    ynand = ~(a & b);  // NAND Operation
    ynor = ~(a | b);   // NOR Operation
    yxor = a ^ b;      // XOR Operation
    yxnor =~(a^b);     // XNOR Operation
end
endmodule
```

Gate-level model (allgate_GL.v):

```
module allgate_GL ( a, b, yand,yor,ynot,ynand,ynor,yxor,yxnor );
input a,b;
output yand, yor, ynot, ynand, ynor, yxor, yxnor;
and  G1(yand,a,b);    // AND Operation
or   G2(yor,a, b);    // OR Operation
not  G3(ynot,a) ;     // NOT Operation
nand G4(ynand,a,b);   // NAND Operation
nor  G5(ynor,a,b);    // NOR Operation
xor  G6(yxor,a,b);    // XOR Operation
xnor G7(yxnor,a,b);   // XNOR Operation
endmodule
```

Top-level module (lab1_ex1.v), shown for the dataflow model:

```
// [FIX] LEDG[6] was added for the XNOR output (it was left unconnected in the
manual)
module lab1_ex1(SW,LEDG,LEDR);
input[17:0] SW;
output[7:0] LEDG;
output[17:0] LEDR;
assign LEDR=SW;
allgate_DF
DUT(SW[1],SW[2],LEDG[5],LEDG[4],LEDG[3],LEDG[2],LEDG[1],LEDG[0],LEDG[6]);
endmodule
```

For the behavioral and gate-level models, only the module name in the instantiation was changed to `allgate_BH` and `allgate_GL`, respectively.

Testbench (tb_allgate_DF.v):

```
`timescale 1ns/1ps
// Testbench - Lab 1 Experiment 1 (basic logic gates) - allgate_DF
module tb_allgate_DF;
reg a, b;
wire yand, yor, ynot, ynand, ynor, yxor, yxnor;
integer i, errors;

allgate_DF DUT(.a(a), .b(b), .yand(yand), .yor(yor), .ynot(ynot), .ynand(ynand),
        .ynor(ynor), .yxor(yxor), .yxnor(yxnor));

initial begin
    errors = 0;
    $display(" a b | AND OR NOT NAND NOR XOR XNOR");
    for (i = 0; i < 4; i = i + 1) begin
        {a, b} = i;
        #10;
        $display(" %b %b |  %b   %b   %b    %b    %b   %b    %b",
```

```
                a, b, yand, yor, ynot, ynand, ynor, yxor, yxnor);
        if ({yand,yor,ynot,ynand,ynor,yxor,yxnor} !==
            {a&b, a|b, ~a, ~(a&b), ~(a|b), a^b, ~(a^b)}) errors = errors + 1;
    end
    if (errors == 0) $display("RESULT: PASSED - all 4 cases correct");
    else             $display("RESULT: FAILED - %0d wrong cases", errors);
    $stop;
end
endmodule
```

The testbenches of the other two models are identical except for the name of the instantiated module.

2.2 Experiment 2: Half Adder

The half adder adds the two bits A (SW[1]) and B (SW[2]). The sum S is displayed on LEDG[0] and the carry Cout on LEDG[1]. The reference code of all three models was used without modification; only the top-level modules were written.

Dataflow model (half_adder_dataflow.v):

```
module half_adder_dataflow(input a, b, output s, Cout);
assign s = a ^ b;
assign Cout = a & b;
endmodule
```

Behavioral model (half_adder_behavior.v):

```
module half_adder_behavior(sum,carry,a,b );
output sum,carry;
input a,b;
reg sum,carry;
always @(a,b)
begin
sum <= a ^ b;
carry <= a&b ;
end
endmodule
```

Gate-level model (half_adder_structeral.v):

```
module half_adder_structeral(input a, b, output s, Cout);
xor G1(s,a,b);
and G2(Cout,a,b);
endmodule
```

Top-level module (lab1_ex2.v):

```
module lab1_ex2(SW,LEDG,LEDR);
input[17:0] SW;
output[7:0] LEDG;
output[17:0] LEDR;
```

```
assign LEDR=SW;
half_adder_dataflow DUT(SW[1],SW[2],LEDG[0],LEDG[1]);      // (a,b,s,Cout)
// Behavioral model (ports: sum,carry,a,b):
//   half_adder_behavior DUT(LEDG[0],LEDG[1],SW[1],SW[2]);
// Gate-level model (ports: a,b,s,Cout):
//   half_adder_structeral DUT(SW[1],SW[2],LEDG[0],LEDG[1]);
endmodule
```

Note that the behavioral module declares its outputs before its inputs, so the port order in its instantiation differs from the other two models.

Testbench (tb_half_adder_dataflow.v):

```
`timescale 1ns/1ps
// Testbench - Lab 1 Experiment 2 (half adder) - half_adder_dataflow
module tb_half_adder_dataflow;
reg A, B;
wire S, Cout;
integer i, errors;

half_adder_dataflow DUT(.a(A), .b(B), .s(S), .Cout(Cout));

initial begin
    errors = 0;
    $display(" A B | Cout S");
    for (i = 0; i < 4; i = i + 1) begin
        {A, B} = i;
        #10;
        $display(" %b %b |   %b  %b", A, B, Cout, S);
        if ({Cout, S} !== A + B) errors = errors + 1;
    end
    if (errors == 0) $display("RESULT: PASSED - all 4 cases correct");
    else             $display("RESULT: FAILED - %0d wrong cases", errors);
    $stop;
end
endmodule
```

The testbenches of the other two models are identical except for the name of the instantiated module and its port names.

2.3 Experiment 3: Full Adder

The full adder adds A (SW[1]), B (SW[2]) and Cin (SW[3]). S is displayed on LEDG[0] and Cout on LEDG[1]. The lab manual provides one dataflow model, three behavioral variants and one structural model. As required, three models were implemented: the dataflow model, the behavioral model using `case`, and the structural model.

Dataflow model (full_adder_DF.v):

```
module full_adder_DF(input a, b, cin, output S, Cout);
assign S = a ^ b ^ cin;
```

```
assign Cout = (a & b) | (b & cin) | (a & cin);
endmodule
```

Behavioral model using case (full_adder_BH1.v):

```
// [FIX 1] The keyword 'module' was added (missing in the manual)
module full_adder_BH1 (input wire A, B, Cin, output reg S, output reg Cout);
always @(A or B or Cin)
begin
// [FIX 2] case (A | B | Cin) -> case ({A,B,Cin})
//         '|' is a bitwise OR (1-bit result); {} concatenates the inputs into a
3-bit value
case ({A,B,Cin})
3'b000: begin S = 0; Cout = 0; end
3'b001: begin S = 1; Cout = 0; end
3'b010: begin S = 1; Cout = 0; end
3'b011: begin S = 0; Cout = 1; end
3'b100: begin S = 1; Cout = 0; end
3'b101: begin S = 0; Cout = 1; end
3'b110: begin S = 0; Cout = 1; end
3'b111: begin S = 1; Cout = 1; end
endcase
end
endmodule
// [FIX 3] The keyword 'endmodule' was added (missing in the manual)
```

Structural model (full_adder_STRU.v):

```
// [FIX 1] Port list (a,b,cin,Cum,Carry) -> (A,B,Cin,Sum,Carry) to match the
declarations
//         (Verilog is case-sensitive and 'Cum' was a typing error)
module full_adder_STRU(A,B,Cin,Sum,Carry);
output Sum,Carry;
input A,B,Cin;
wire x,y,z;
xor g1(x,A,B);
xor g2(Sum,x,Cin);
and g3(y,x,Cin);
and g4(z,A,B);
// [FIX 2] or g5(Carry,x,y) -> or g5(Carry,y,z)   (Carry = Y + Z, see Figure 3)
or g5(Carry,y,z);
endmodule
```

Top-level module (lab1_ex3.v):

```
module lab1_ex3(SW,LEDG,LEDR);
input[17:0] SW;
output[7:0] LEDG;
output[17:0] LEDR;
assign LEDR=SW;
full_adder_DF DUT(SW[1],SW[2],SW[3],LEDG[0],LEDG[1]);  // (A,B,Cin,S,Cout)
// Other models: full_adder_BH1 / full_adder_STRU with the same port order
endmodule
```

Testbench (tb_full_adder_DF.v):

```
`timescale 1ns/1ps
```

```
// Testbench - Lab 1 Experiment 3 (full adder) - full_adder_DF
module tb_full_adder_DF;
reg A, B, Cin;
wire S, Cout;
integer i, errors;

full_adder_DF DUT(.a(A), .b(B), .cin(Cin), .S(S), .Cout(Cout));

initial begin
    errors = 0;
    $display(" A B Cin | Cout S");
    for (i = 0; i < 8; i = i + 1) begin
        {A, B, Cin} = i;
        #10;
        $display(" %b %b  %b  |   %b  %b", A, B, Cin, Cout, S);
        if ({Cout, S} !== A + B + Cin) errors = errors + 1;
    end
    if (errors == 0) $display("RESULT: PASSED - all 8 cases correct");
    else             $display("RESULT: FAILED - %0d wrong cases", errors);
    $stop;
end
endmodule
```

The testbenches of the other two models are identical except for the name of the instantiated module and its port names.

2.4 Experiment 4: 4-bit Ripple Carry Adder

The 4-bit adder computes A + B + Cin. Its I/O assignment is given in Table 9. The structural and dataflow models are taken from the lab manual; the structural model reuses the corrected `full_adder_STRU` module of Experiment 3. The manual does not provide a behavioral model, so one was written: it describes the ripple-carry behavior with a `for` loop that adds the operands bit by bit, starting from bit 0, and passes the carry to the next bit.

Table 9: *I/O assignment of Experiment 4.*

Signal	Board I/O	Signal	Board I/O
A[3:0]	SW[3:0]	Sum[3:0]	LEDG[3:0]
B[3:0]	SW[7:4]	Cout	LEDG[4]
Cin	SW[8]		

Structural model (four_bit_adder_STRU.v, uses full_adder_STRU.v of Experiment 3):

```
module four_bit_adder_STRU(cin,a,b,s,cout);
input [3:0] a,b;
input cin;
output [3:0] s;
output cout;
wire [2:0] w_carry;
```

```
full_adder_STRU C1(a[0],b[0],cin,s[0], w_carry[0]);
full_adder_STRU C2(a[1],b[1], w_carry[0],s[1], w_carry[1]);
full_adder_STRU C3(a[2],b[2], w_carry[1],s[2], w_carry[2]);
full_adder_STRU C4(a[3],b[3], w_carry[2],s[3], cout);
endmodule
```

Dataflow model (Four_bit_Adder_DF.v):

```
module Four_bit_Adder_DF(A,B,Cin,Sum,Cout);
input [3:0] A,B;
input Cin;
output wire [3:0]Sum;
output wire Cout;
wire [4:0]temp;
assign temp=A+B+Cin;
assign Sum=temp[3:0];
assign Cout=temp[4];
endmodule
```

Behavioral model (Four_bit_Adder_BH.v, written for this report):

```
// Ripple-carry behavior: add bit by bit from bit 0 to bit 3,
// the carry c of each bit is used by the next bit.
module Four_bit_Adder_BH(A,B,Cin,Sum,Cout);
input [3:0] A,B;
input Cin;
output reg [3:0] Sum;
output reg Cout;
reg c;
integer i;
always @(A or B or Cin)
begin
    c = Cin;
    for (i = 0; i < 4; i = i + 1)
    begin
        Sum[i] = A[i] ^ B[i] ^ c;
        c = (A[i] & B[i]) | (A[i] & c) | (B[i] & c);
    end
    Cout = c;
end
endmodule
```

Top-level module (lab1_ex4.v):

```
module lab1_ex4(SW,LEDG,LEDR);
input[17:0] SW;
output[7:0] LEDG;
output[17:0] LEDR;
assign LEDR=SW;
Four_bit_Adder_DF DUT(SW[3:0],SW[7:4],SW[8],LEDG[3:0],LEDG[4]); //
(A,B,Cin,Sum,Cout)
// Behavioral:
//   Four_bit_Adder_BH DUT(SW[3:0],SW[7:4],SW[8],LEDG[3:0],LEDG[4]);
// Structural (ports: cin,a,b,s,cout):
//   four_bit_adder_STRU DUT(SW[8],SW[3:0],SW[7:4],LEDG[3:0],LEDG[4]);
endmodule
```

Testbench (tb_Four_bit_Adder_DF.v):

```
`timescale 1ns/1ps
// Testbench - Lab 1 Experiment 4 (4-bit ripple carry adder) - Four_bit_Adder_DF
module tb_Four_bit_Adder_DF;
reg  [3:0] A, B;
reg        Cin;
wire [3:0] Sum;
wire       Cout;
integer errors;

Four_bit_Adder_DF DUT(.A(A), .B(B), .Cin(Cin), .Sum(Sum), .Cout(Cout));

task check; input [3:0] ta, tb; input tc;
begin
    A = ta; B = tb; Cin = tc; #10;
    $display(" %2d + %2d + %b = %2d   Cout=%b Sum=%b", A, B, Cin, {Cout, Sum},
Cout, Sum);
    if ({Cout, Sum} !== A + B + Cin) errors = errors + 1;
end
endtask

initial begin
    errors = 0;
    $display("  A +  B +Cin = result");
    check(4'd0,  4'd0,  1'b0);
    check(4'd4,  4'd4,  1'b0);
    check(4'd10, 4'd10, 1'b0);
    check(4'd15, 4'd15, 1'b0);
    check(4'd15, 4'd0,  1'b1);
    check(4'd2,  4'd10, 1'b0);
    check(4'd5,  4'd8,  1'b0);
    check(4'd5,  4'd4,  1'b0);
    check(4'd7,  4'd9,  1'b1);
    check(4'd8,  4'd8,  1'b0);
    if (errors == 0) $display("RESULT: PASSED - all cases correct");
    else             $display("RESULT: FAILED - %0d wrong cases", errors);
    $stop;
end
endmodule
```

The testbenches of the other two models are identical except for the name of the instantiated module and its port names.

2.5 Experiment 5: 2:1 Multiplexer

The 2:1 MUX has data inputs i0 (SW[1]) and i1 (SW[2]), select input sel (SW[3]) and output y (LEDG[0]). The reference code of all three models was used without modification.

Dataflow model (mux21_data_flow.v):

```
module mux21_data_flow(i0,i1,sel,y);
input i0,i1,sel;
output y;
assign y =(i0&(~sel))|(i1&sel);
endmodule
```

Behavioral model (mux21_Behavioural.v):

```
module mux21_Behavioural (i0,i1,sel,y);
input i0,i1,sel;
output y;
reg y;
always@(*)
begin
if(sel==0) y=i0;
if(sel==1)y=i1;
end
endmodule
```

Structural model (mux21_structural.v):

```
module mux21_structural(i0,i1,sel,y);
input i0,i1,sel;
output y;
wire net1,net2,net3;
not g1(net1,sel);
and g2(net2,i1,sel);
and g3(net3,i0,net1);
or g4(y,net3,net2);
endmodule
```

Top-level module (lab1_ex5.v):

```
module lab1_ex5(SW,LEDG,LEDR);
input[17:0] SW;
output[7:0] LEDG;
output[17:0] LEDR;
assign LEDR=SW;
mux21_data_flow DUT(SW[1],SW[2],SW[3],LEDG[0]);  // (i0,i1,sel,y)
// Other models: mux21_Behavioural / mux21_structural with the same port order
endmodule
```

Testbench (tb_mux21_data_flow.v):

```
`timescale 1ns/1ps
// Testbench - Lab 1 Experiment 5 (2:1 MUX) - mux21_data_flow
module tb_mux21_data_flow;
reg i0, i1, sel;
wire y;
integer i, errors;


mux21_data_flow DUT(.i0(i0), .i1(i1), .sel(sel), .y(y));


initial begin
    errors = 0;
    $display(" sel i1 i0 | y");
    for (i = 0; i < 8; i = i + 1) begin
        {sel, i1, i0} = i;
        #10;
        $display("  %b   %b  %b | %b", sel, i1, i0, y);
        if (y !== (sel ? i1 : i0)) errors = errors + 1;
    end
```

```
    if (errors == 0) $display("RESULT: PASSED - all 8 cases correct");
    else             $display("RESULT: FAILED - %0d wrong cases", errors);
    $stop;
end
endmodule
```

The testbenches of the other two models are identical except for the name of the instantiated module.

2.6 Experiment 6: 4:1 Multiplexer

The 4:1 MUX has data inputs i0–i3 (SW[0]–SW[3]), select inputs s0 (SW[4]) and s1 (SW[5]) and output y (LEDG[0]). The lab manual provides a structural model, a dataflow model and two behavioral models. A simulation of all four modules showed that they do not use the same select convention: the structural model, the dataflow model and `mux41beh_v2` treat s1 as the most significant select bit, whereas `mux41beh_v1` treats s[0] as the most significant bit and therefore swaps inputs i1 and i2 (Table 10). To keep the three implemented models consistent, `mux41beh_v2` was chosen as the behavioral model, so no reference code had to be modified.

Table 10: *Input selected by the reference 4:1 MUX modules.*

s1 s0	Structural	Dataflow	mux41beh_v1	mux41beh_v2
00	i0	i0	i0	i0
01	i1	i1	**i2**	i1
10	i2	i2	**i1**	i2
11	i3	i3	i3	i3

Structural model (mux41_strutural.v):

```
module mux41_strutural (i0,i1,i2,i3,s0,s1,y);
input i0,i1,i2,i3,s0,s1;
output y;
wire n_s0,n_s1, a0,a1,a2,a3;
not g0(n_s0,s0);
not g1(n_s1,s1);
and g2(a0,i0,n_s0,n_s1);
and g3(a1,i1,n_s1,s0);
and g4(a2,i2,s1,n_s0);
and g5(a3,i3,s1,s0);
or g6(y,a0,a1,a2,a3);
endmodule
```

Dataflow model (mux41_df.v):

```
module mux41_df (i0,i1,i2,i3,s0,s1,y);
input i0,i1,i2,i3,s0,s1;
```

```
output y;
assign y= i0&(~s1)&(~s0) | i1 &(~s1)&s0 | i2&s1&(~s0) | i3&s1&s0;
endmodule
```

Behavioral model (mux41beh_v2.v):

```
module mux41beh_v2(in,s,y );
output y ;
input [3:0]in ;
input [1:0]s ;
reg y;
always@(in,s)
begin
case ({s[1],s[0]})
2'b00: y <= in[0];
2'b01: y <= in[1];
2'b10: y <= in[2];
2'b11: y <= in[3];
endcase
end
endmodule
```

Top-level module (lab1_ex6.v):

```
module lab1_ex6(SW,LEDG,LEDR);
input[17:0] SW;
output[7:0] LEDG;
output[17:0] LEDR;
assign LEDR=SW;
// ports: (i0,i1,i2,i3,s0,s1,y)
mux41_df DUT(SW[0],SW[1],SW[2],SW[3],SW[4],SW[5],LEDG[0]);
// Structural: mux41_strutural with the same port order
// Behavioral: mux41beh_v2 DUT(SW[3:0],SW[5:4],LEDG[0]);   // (in,s,y)
endmodule
```

Testbench (tb_mux41_df.v):

```
`timescale 1ns/1ps
// Testbench - Lab 1 Experiment 6 (4:1 MUX) - mux41_df
// Convention: s[1] = s1 = MSB, s[0] = s0 = LSB
module tb_mux41_df;
reg  [3:0] in;
reg  [1:0] s;
wire y;
integer k, errors;

mux41_df DUT(.i0(in[0]), .i1(in[1]), .i2(in[2]), .i3(in[3]), .s0(s[0]),
.s1(s[1]), .y(y));

initial begin
    errors = 0;
    $display(" s1 s0 | i3 i2 i1 i0 | y");
    // For each select value: only the selected input = 1 (y = 1),
    // then only the selected input = 0 (y = 0)
    for (k = 0; k < 4; k = k + 1) begin
        s = k;
        in = 4'b0001 << k;    #10;
        $display("  %b  %b |  %b  %b  %b  %b | %b", s[1], s[0], in[3], in[2],
```

```
in[1], in[0], y);
        if (y !== 1'b1) errors = errors + 1;
        in = ~(4'b0001 << k); #10;
        $display("  %b  %b |  %b  %b  %b  %b | %b", s[1], s[0], in[3], in[2],
in[1], in[0], y);
        if (y !== 1'b0) errors = errors + 1;
    end
    if (errors == 0) $display("RESULT: PASSED - all 8 cases correct");
    else             $display("RESULT: FAILED - %0d wrong cases", errors);
    $stop;
end
endmodule
```

The testbenches of the other two models are identical except for the name of the instantiated module and its port names.

2.7 Experiment 7: 16:1 Multiplexer from Five 4:1 Multiplexers

The lab manual gives only the block diagram (Figure 7) and the truth table of this circuit, so the structural model was written for this report. It instantiates the 4:1 MUX module `mux41_strutural` of Experiment 6 five times: four first-level instances M0–M3 share S1 S0, and the second-level instance M4 uses S3 S2.

The circuit needs 16 data inputs and 4 select inputs, i.e. 20 inputs, but the DE2 board has only 18 slide switches. Therefore two push-buttons were used for the last two data inputs (Table 11). The push-buttons are active-low (pressed = 0), so they are inverted in the top-level module.

Table 11: *I/O assignment of Experiment 7.*

Signal	Board I/O
S3 S2 S1 S0	SW[17] SW[16] SW[15] SW[14]
D0–D13	SW[0]–SW[13]
D14	KEY[0] (pressed = 1)
D15	KEY[1] (pressed = 1)
y	LEDG[0]

Structural model (mux161_structural.v, written for this report; uses mux41_strutural.v):

```
// 16:1 MUX built from five 4:1 MUX modules
// Level 1: four 4:1 MUXes select within each group of 4 inputs (s1 s0)
// Level 2: one 4:1 MUX selects one of the level-1 outputs (s3 s2)
module mux161_structural(d, s, y);
input [15:0] d;     // d[0]..d[15] = D0..D15
input [3:0]  s;     // s[3] = S3 (MSB) ... s[0] = S0 (LSB)
output y;
wire [3:0] m;       // outputs of the level-1 multiplexers
```

```
mux41_strutural M0(d[0],  d[1],  d[2],  d[3],  s[0], s[1], m[0]);  // D0 - D3
mux41_strutural M1(d[4],  d[5],  d[6],  d[7],  s[0], s[1], m[1]);  // D4 - D7
mux41_strutural M2(d[8],  d[9],  d[10], d[11], s[0], s[1], m[2]);  // D8 - D11
mux41_strutural M3(d[12], d[13], d[14], d[15], s[0], s[1], m[3]);  // D12 - D15

mux41_strutural M4(m[0], m[1], m[2], m[3], s[2], s[3], y);         // level 2
endmodule
```

Top-level module (lab1_ex7.v):

```
// The DE2 board has only 18 switches; 20 inputs are needed
//   S3 S2 S1 S0 = SW17 SW16 SW15 SW14
//   D0 .. D13   = SW0 .. SW13
//   D14 = KEY0, D15 = KEY1 (push-buttons are active-low, so they are inverted)
//   y = LEDG0
module lab1_ex7(SW,KEY,LEDG,LEDR);
input[17:0] SW;
input[3:0] KEY;
output[7:0] LEDG;
output[17:0] LEDR;
assign LEDR=SW;
mux161_structural DUT({~KEY[1],~KEY[0],SW[13:0]}, SW[17:14], LEDG[0]); // (d,s,y)
endmodule
```

Testbench (tb_mux161_structural.v):

```
`timescale 1ns/1ps
// Testbench - Lab 1 Experiment 7 (16:1 MUX from five 4:1 MUX) -
mux161_structural
module tb_mux161_structural;
reg  [15:0] d;
reg  [3:0]  s;
wire y;
integer k, errors;

mux161_structural DUT(.d(d), .s(s), .y(y));

initial begin
    errors = 0;
    $display(" S3S2S1S0 | sel. | y (sel. = 1) | y (sel. = 0)");
    for (k = 0; k < 16; k = k + 1) begin
        s = k;
        d = 16'h0001 << k;    #10;
        if (y !== 1'b1) errors = errors + 1;
        $write("   %b   | D%0d\t |      %b", s, k, y);
        d = ~(16'h0001 << k); #10;
        if (y !== 1'b0) errors = errors + 1;
        $display("       |      %b", y);
    end
    if (errors == 0) $display("RESULT: PASSED - all 32 cases correct");
    else             $display("RESULT: FAILED - %0d wrong cases", errors);
    $stop;
end
endmodule
```

3 Experimental Results

All 19 models compiled successfully with 0 errors and were programmed into the FPGA. For each model the test combinations listed below were applied with the switches, and the LED outputs were recorded. In the tables, 1 means that the LED is on, and "✓" means that the observed output of the model is identical to the expected output. The simulation results are presented at the end of each experiment: all 19 testbenches reported PASSED, i.e. the simulated outputs matched the expected values for every applied input combination.

3.1 Experiment 1

Table 12 lists the outputs observed for the four input combinations. The LEDs that were on are LEDG6, 3, 2, 1 for ab = 00; LEDG4, 3, 2, 0 for ab = 01; LEDG4, 2, 0 for ab = 10; and LEDG6, 5, 4 for ab = 11. The DF, BH and GL models produced identical LED outputs; therefore, one set of photographs is shown for this experiment.

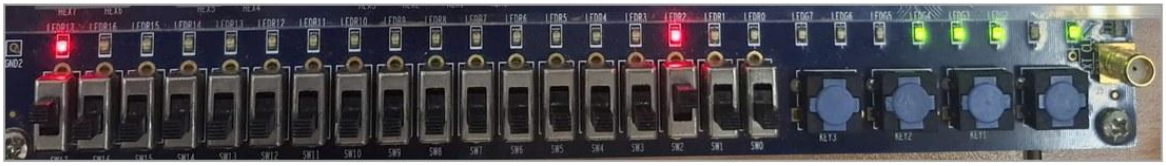
Table 12: *Experiment 1 – expected outputs and observed results.*

a (SW1)	b (SW2)	AND	OR	NOT	NAND	NOR	XOR	XNOR	DF	BH	GL
0	0	0	0	1	1	1	0	1	✓	✓	✓
0	1	0	1	1	1	0	1	0	✓	✓	✓
1	0	0	1	0	1	0	1	0	✓	✓	✓
1	1	1	1	0	0	0	0	1	✓	✓	✓

Figure 8 shows three of these combinations on the board (rows 3, 2 and 4 of Table 12): in (a) OR, NAND and XOR are on; in (b) OR, NOT a, NAND and XOR are on; in (c) XNOR, AND and OR are on. SW17 is not used in this experiment.



(a) a = 1, b = 0: LEDG4, 2, 0 on



(b) a = 0, b = 1: LEDG4, 3, 2, 0 on

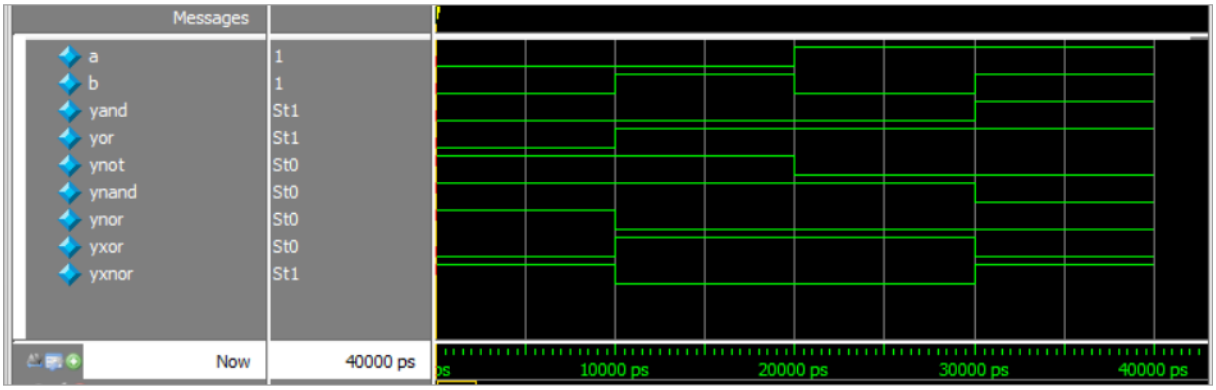


(c) a = 1, b = 1: LEDG6, 5, 4 on

Figure 8: *FPGA implementation results of Experiment 1.*

Figure 9 shows the simulation results: (a) the waveform of the dataflow model, in which each input combination is applied for 10 ns, and (b)–(d) the transcripts of the three models, in which the testbench compares the seven outputs with the expected Boolean values and reports PASSED.

(a) Waveform (dataflow model)



(b) Transcript – dataflow model

```
VSIM 14> run -all
# a b | AND OR NOT NAND NOR XOR XNOR
# 0 0 | 0  0  1  1  1  0  1
# 0 1 | 0  1  1  1  0  1  0
# 1 0 | 0  1  0  1  0  1  0
# 1 1 | 1  1  0  0  0  0  1
# RESULT: PASSED - all 4 cases correct
# Break in Module tb_allgate_DF at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex1_DF/tb_allgate_DF.v line 24
```

(c) Transcript – behavioral model

```
VSIM 17> run -all
# a b | AND OR NOT NAND NOR XOR XNOR
# 0 0 | 0  0  1  1  1  0  1
# 0 1 | 0  1  1  1  0  1  0
# 1 0 | 0  1  0  1  0  1  0
# 1 1 | 1  1  0  0  0  0  1
# RESULT: PASSED - all 4 cases correct
# Break in Module tb_allgate_BH at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex1_BH/tb_allgate_BH.v line 24
```

(d) Transcript – gate-level model

```
VSIM 20> run -all
# a b | AND OR NOT NAND NOR XOR XNOR
# 0 0 | 0  0  1  1  1  0  1
# 0 1 | 0  1  1  1  0  1  0
# 1 0 | 0  1  0  1  0  1  0
# 1 1 | 1  1  0  0  0  0  1
# RESULT: PASSED - all 4 cases correct
# Break in Module tb_allgate_GL at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex1_GL/tb_allgate_GL.v line 24
```

Figure 9: *Simulation results of Experiment 1.*

3.2 Experiment 2

Table 13 lists the results of the half adder. LEDG0 (S) is on for 01 and 10, and LEDG1 (Cout) is on only for 11. The DF, BH and GL models produced identical LED outputs; therefore, one set of photographs is shown for this experiment.

Table 13: *Experiment 2 – expected outputs and observed results.*

A (SW1)	B (SW2)	Cout (LEDG1)	S (LEDG0)	DF	BH	GL
0	0	0	0	✓	✓	✓
1	0	0	1	✓	✓	✓
0	1	0	1	✓	✓	✓
1	1	1	0	✓	✓	✓

Figure 10 shows rows 2, 3 and 4 of Table 13 on the board; S is displayed on LEDG0 and Cout on LEDG1. SW17 is not used in this experiment.



(a) A = 1, B = 0: Cout S = 0 1



(b) A = 0, B = 1: Cout S = 0 1

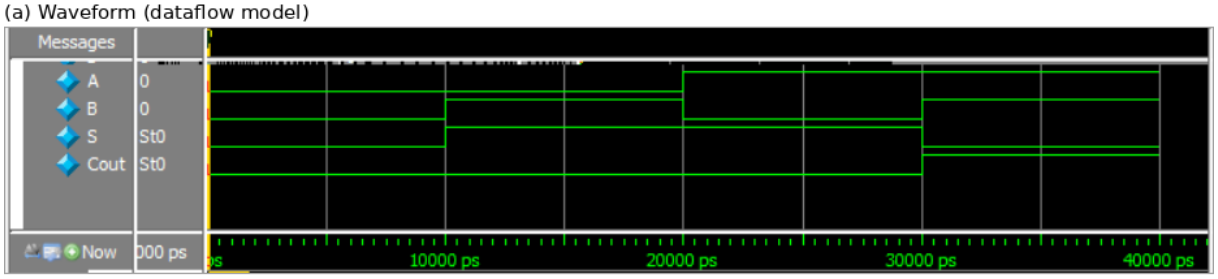


(c) A = 1, B = 1: Cout S = 1 0

Figure 10: *FPGA implementation results of Experiment 2.*

Figure 11 shows the simulation results: (a) the waveform of the dataflow model, in which each input combination is applied for 10 ns, and (b)–(d) the transcripts of the three models, in which the testbench compares {Cout, S} with A + B and reports PASSED.

(a) Waveform (dataflow model)



(b) Transcript – dataflow model

```
# 0 1 |  0  1
# 1 0 |  0  1
# 1 1 |  1  0
# RESULT: PASSED - all 4 cases correct
# Break in Module tb_half_adder_dataflow at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex2_DF/tb_half_adder_dataflow.v line 21
```

(c) Transcript – behavioral model

```
VSIM 7> run -all
#  A B | Cout S
# 0 0 |  0  0
# 0 1 |  0  1
# 1 0 |  0  1
# 1 1 |  1  0
# RESULT: PASSED - all 4 cases correct
# Break in Module tb_half_adder_behavior at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex2_BH/tb_half_adder_behavior.v line 21
```

(d) Transcript – gate-level model

```
VSIM 10> run -all
#  A B | Cout S
# 0 0 |  0  0
# 0 1 |  0  1
# 1 0 |  0  1
# 1 1 |  1  0
# RESULT: PASSED - all 4 cases correct
# Break in Module tb_half_adder_structeral at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex2_GL/tb_half_adder_structeral.v line 21
```

Figure 11: *Simulation results of Experiment 2.*

3.3 Experiment 3

Table 14 lists the results of the full adder for all eight input combinations. With only Cin = 1, LEDG0 (S) was on. With A = B = 1 and Cin = 0, LEDG1 (Cout) was on and LEDG0 was off. The DF, BH and STRU models produced identical LED outputs; therefore, one set of photographs is shown for this experiment.

Table 14: *Experiment 3 – expected outputs and observed results.*

Cin (SW3)	B (SW2)	A (SW1)	Cout	S	DF	BH	STRU
0	0	0	0	0	✓	✓	✓
0	0	1	0	1	✓	✓	✓
0	1	0	0	1	✓	✓	✓
0	1	1	1	0	✓	✓	✓
1	0	0	0	1	✓	✓	✓
1	0	1	1	0	✓	✓	✓
1	1	0	1	0	✓	✓	✓
1	1	1	1	1	✓	✓	✓

Figure 12 shows rows 4, 7 and 8 of Table 14 on the board; S is displayed on LEDG0 and Cout on LEDG1. SW17 is not used in this experiment.



(a) Cin B A = 0 1 1: Cout S = 1 0 (sum = 2)



(b) Cin B A = 1 1 0: Cout S = 1 0 (sum = 2)

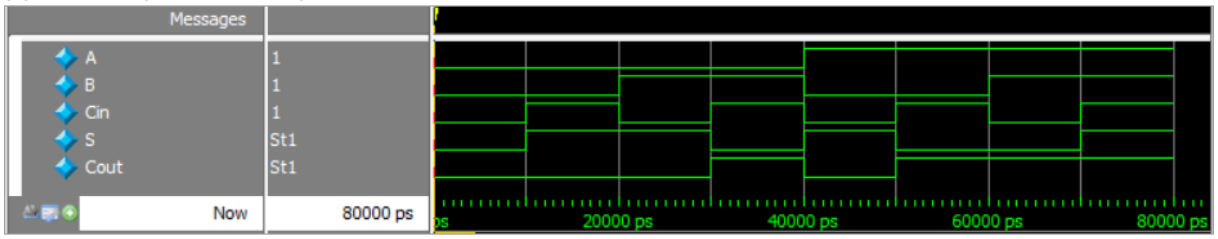


(c) Cin B A = 1 1 1: Cout S = 1 1 (sum = 3)

Figure 12: *FPGA implementation results of Experiment 3.*

Figure 13 shows the simulation results: (a) the waveform of the dataflow model, in which each input combination is applied for 10 ns, and (b)–(d) the transcripts of the three models, in which the testbench compares {Cout, S} with A + B + Cin and reports PASSED.

(a) Waveform (dataflow model)



(b) Transcript – dataflow model

```
VSIM 24> run -all
#  A B Cin | Cout S
# 0 0 0 |  0 0
# 0 0 1 |  0 1
# 0 1 0 |  0 1
# 0 1 1 |  1 0
# 1 0 0 |  0 1
# 1 0 1 |  1 0
# 1 1 0 |  1 0
# 1 1 1 |  1 1
# RESULT: PASSED - all 8 cases correct
# Break in Module tb_full_adder_DF at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex3_DF/tb_full_adder_DF.v line 21
```

(c) Transcript – behavioral model (case)

```
VSIM 27> run -all
#  A B Cin | Cout S
# 0 0 0 |  0 0
# 0 0 1 |  0 1
# 0 1 0 |  0 1
# 0 1 1 |  1 0
# 1 0 0 |  0 1
# 1 0 1 |  1 0
# 1 1 0 |  1 0
# 1 1 1 |  1 1
# RESULT: PASSED - all 8 cases correct
# Break in Module tb_full_adder_BH1 at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex3_BH1/tb_full_adder_BH1.v line 21
```

(d) Transcript – structural model

```
VSIM 2> run -all
#  A B Cin | Cout S
# 0 0 0 |  0 0
# 0 0 1 |  0 1
# 0 1 0 |  0 1
# 0 1 1 |  1 0
# 1 0 0 |  0 1
# 1 0 1 |  1 0
# 1 1 0 |  1 0
# 1 1 1 |  1 1
# RESULT: PASSED - all 8 cases correct
# Break in Module tb_full_adder_STRU at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex3_STRU/tb_full_adder_STRU.v line 21
```

Figure 13: *Simulation results of Experiment 3.*

3.4 Experiment 4

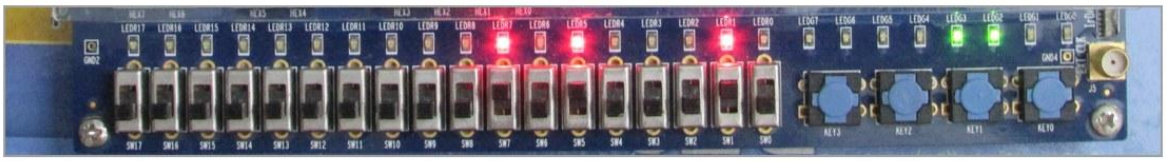
Table 15 lists the additions that were tested. Rows 1–4 are taken from the truth table of the lab manual, row 5 checks the carry input and the propagation of the carry through all four stages, and rows 6–8 are additional random additions. The sum is read from {LEDG4,

LEDG3–LEDG0} as a 5-bit binary number. The DF, STRU and BH models produced identical LED outputs; therefore, one set of photographs is shown for this experiment.

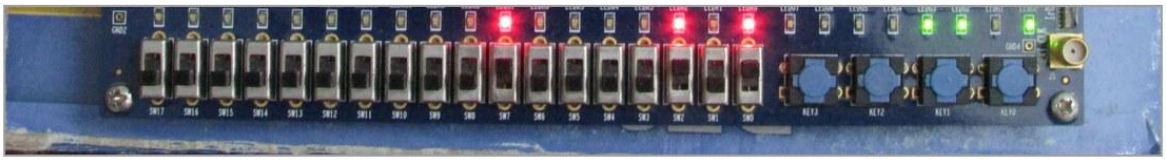
Table 15: *Experiment 4 – expected outputs and observed results.*

A	B	Cin	A + B + Cin	Cout S3S2S1S0	LEDs on	DF	STRU	BH
0000 (0)	0000 (0)	0	0	0 0000	none	✓	✓	✓
0100 (4)	0100 (4)	0	8	0 1000	LEDG3	✓	✓	✓
1010 (10)	1010 (10)	0	20	1 0100	LEDG4, 2	✓	✓	✓
1111 (15)	1111 (15)	0	30	1 1110	LEDG4, 3, 2, 1	✓	✓	✓
1111 (15)	0000 (0)	1	16	1 0000	LEDG4	✓	✓	✓
0010 (2)	1010 (10)	0	12	0 1100	LEDG3, 2	✓	✓	✓
0101 (5)	1000 (8)	0	13	0 1101	LEDG3, 2, 0	✓	✓	✓
0101 (5)	0100 (4)	0	9	0 1001	LEDG3, 0	✓	✓	✓

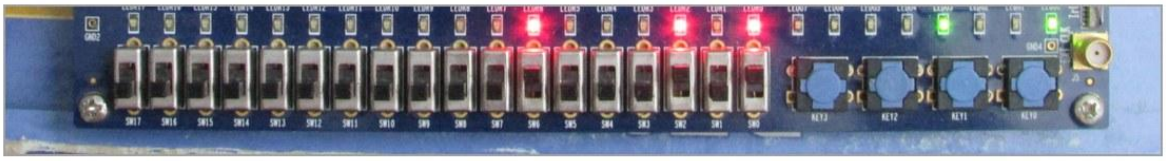
Figure 14 shows the additions of rows 6–8 of Table 15; A is set with SW3–SW0, B with SW7–SW4, and the sum is read from LEDG4–LEDG0.



(a) A = 0010 (2), B = 1010 (10): {Cout,S} = 0 1100 (12)



(b) A = 0101 (5), B = 1000 (8): {Cout,S} = 0 1101 (13)



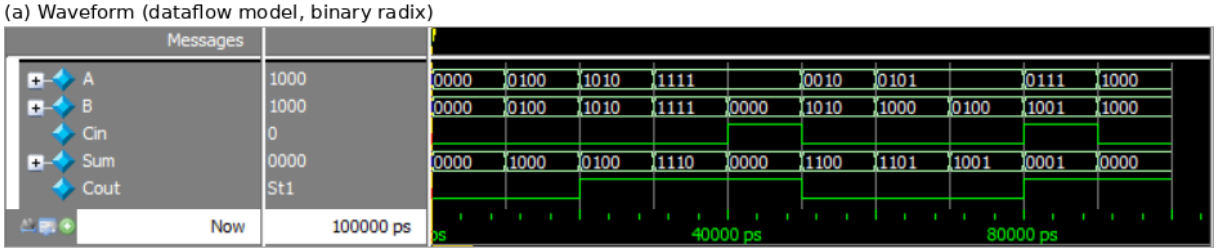
(c) A = 0101 (5), B = 0100 (4): {Cout,S} = 0 1001 (9)

Figure 14: *FPGA implementation results of Experiment 4.*

Figure 15 shows the simulation results: (a) the waveform of the dataflow model (binary radix), in which the ten additions are applied for 10 ns each, and (b)–(d) the

transcripts of the three models, in which the testbench compares {Cout, Sum} with A + B + Cin and reports PASSED.

(a) Waveform (dataflow model, binary radix)



(b) Transcript – dataflow model

```
VSIM 11> run -all
#  A + B +Cin = result
#  0 + 0 + 0 =  0  Cout=0 Sum=0000
#  4 + 4 + 0 =  8  Cout=0 Sum=1000
# 10 + 10 + 0 = 20  Cout=1 Sum=0100
# 15 + 15 + 0 = 30  Cout=1 Sum=1110
# 15 + 0 + 1 = 16  Cout=1 Sum=0000
#  2 + 10 + 0 = 12  Cout=0 Sum=1100
#  5 + 8 + 0 = 13  Cout=0 Sum=1101
#  5 + 4 + 0 =  9  Cout=0 Sum=1001
#  7 + 9 + 1 = 17  Cout=1 Sum=0001
#  8 + 8 + 0 = 16  Cout=1 Sum=0000
# RESULT: PASSED - all cases correct
# Break in Module tb_Four_bit_Adder_DF at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex4_DF/tb_Four_bit_Adder_DF.v line 35
```

(c) Transcript – behavioral model

```
VSIM 14> run -all
#  A + B +Cin = result
#  0 + 0 + 0 =  0  Cout=0 Sum=0000
#  4 + 4 + 0 =  8  Cout=0 Sum=1000
# 10 + 10 + 0 = 20  Cout=1 Sum=0100
# 15 + 15 + 0 = 30  Cout=1 Sum=1110
# 15 + 0 + 1 = 16  Cout=1 Sum=0000
#  2 + 10 + 0 = 12  Cout=0 Sum=1100
#  5 + 8 + 0 = 13  Cout=0 Sum=1101
#  5 + 4 + 0 =  9  Cout=0 Sum=1001
#  7 + 9 + 1 = 17  Cout=1 Sum=0001
#  8 + 8 + 0 = 16  Cout=1 Sum=0000
# RESULT: PASSED - all cases correct
# Break in Module tb_Four_bit_Adder_BH at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex4_BH/tb_Four_bit_Adder_BH.v line 35
```

(d) Transcript – structural model

```
VSIM 17> run -all
#  A + B +Cin = result
#  0 + 0 + 0 =  0  Cout=0 Sum=0000
#  4 + 4 + 0 =  8  Cout=0 Sum=1000
# 10 + 10 + 0 = 20  Cout=1 Sum=0100
# 15 + 15 + 0 = 30  Cout=1 Sum=1110
# 15 + 0 + 1 = 16  Cout=1 Sum=0000
#  2 + 10 + 0 = 12  Cout=0 Sum=1100
#  5 + 8 + 0 = 13  Cout=0 Sum=1101
#  5 + 4 + 0 =  9  Cout=0 Sum=1001
#  7 + 9 + 1 = 17  Cout=1 Sum=0001
#  8 + 8 + 0 = 16  Cout=1 Sum=0000
# RESULT: PASSED - all cases correct
# Break in Module tb_four_bit_adder_STRU at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex4_STRU/tb_four_bit_adder_STRU.v line 35
```

Figure 15: *Simulation results of Experiment 4.*

3.5 Experiment 5

Table 16 lists the results of the 2:1 MUX. In every test the two data inputs have different values, so the LED shows which input is selected. The DF, BH and STRU models

produced identical LED outputs; therefore, one set of photographs is shown for this experiment.

Table 16: *Experiment 5 – expected outputs and observed results.*

sel (SW3)	i1 (SW2)	i0 (SW1)	y (LEDG0)	Selected	DF	BH	STRU
0	0	1	1	i0	✓	✓	✓
0	1	0	0	i0	✓	✓	✓
1	0	1	0	i1	✓	✓	✓
1	1	0	1	i1	✓	✓	✓

Figure 16 shows rows 1, 3 and 4 of Table 16; the red LEDs mirror the switches and the green LEDG0 (right) shows the output y.



(a) sel = 0, i1 = 0, i0 = 1: y = 1 (i0 selected)



(b) sel = 1, i1 = 0, i0 = 1: y = 0 (i1 selected)

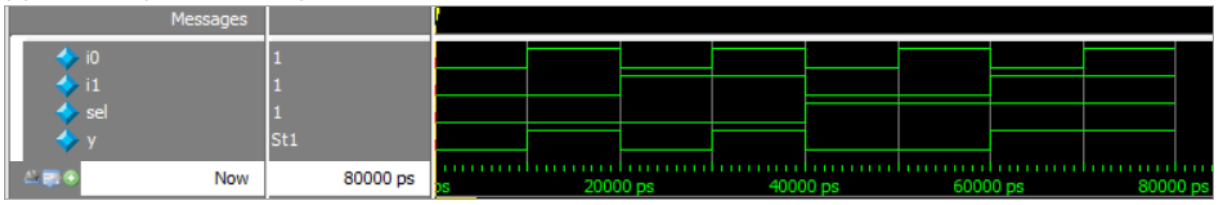


(c) sel = 1, i1 = 1, i0 = 0: y = 1 (i1 selected)

Figure 16: *FPGA implementation results of Experiment 5.*

Figure 17 shows the simulation results: (a) the waveform of the dataflow model, in which each input combination is applied for 10 ns, and (b)–(d) the transcripts of the three models, in which the testbench compares y with (sel ? i1 : i0) and reports PASSED.

(a) Waveform (dataflow model)



(b) Transcript – dataflow model

```
VSIM 21> run -all
#  sel i1 i0 | y
#   0   0  0 | 0
#   0   0  1 | 1
#   0   1  0 | 0
#   0   1  1 | 1
#   1   0  0 | 0
#   1   0  1 | 0
#   1   1  0 | 1
#   1   1  1 | 1
# RESULT: PASSED - all 8 cases correct
# Break in Module tb_mux21_data_flow at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex5_DF/tb_mux21_data_flow.v line 21
```

(c) Transcript – behavioral model

```
VSIM 24> run -all
#  sel i1 i0 | y
#   0   0  0 | 0
#   0   0  1 | 1
#   0   1  0 | 0
#   0   1  1 | 1
#   1   0  0 | 0
#   1   0  1 | 0
#   1   1  0 | 1
#   1   1  1 | 1
# RESULT: PASSED - all 8 cases correct
# Break in Module tb_mux21_Behavioural at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex5_BH/tb_mux21_Behavioural.v line 21
```

(d) Transcript – structural model

```
VSIM 27> run -all
#  sel i1 i0 | y
#   0   0  0 | 0
#   0   0  1 | 1
#   0   1  0 | 0
#   0   1  1 | 1
#   1   0  0 | 0
#   1   0  1 | 0
#   1   1  0 | 1
#   1   1  1 | 1
# RESULT: PASSED - all 8 cases correct
# Break in Module tb_mux21_structural at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex5_STRU/tb_mux21_structural.v line 21
```

Figure 17: *Simulation results of Experiment 5.*

3.6 Experiment 6

For each select value only the selected input was set to 1, and LEDG0 turned on (Table 17). When the selected input was set to 0 and the other three inputs were set to 1, LEDG0 turned off for every select value. The DF, BH and STRU models produced identical LED outputs; therefore, one set of photographs is shown for this experiment.

Table 17: *Experiment 6 – expected outputs and observed results.*

s1 (SW5)	s0 (SW4)	Selected	Input set to 1	y (LEDG0)	DF	BH	STRU
0	0	i0	SW0	1	✓	✓	✓
0	1	i1	SW1	1	✓	✓	✓
1	0	i2	SW2	1	✓	✓	✓
1	1	i3	SW3	1	✓	✓	✓

Figure 18 shows the four rows of Table 17; the red LEDs mirror the switches and the green LEDG0 (right) shows the output y.

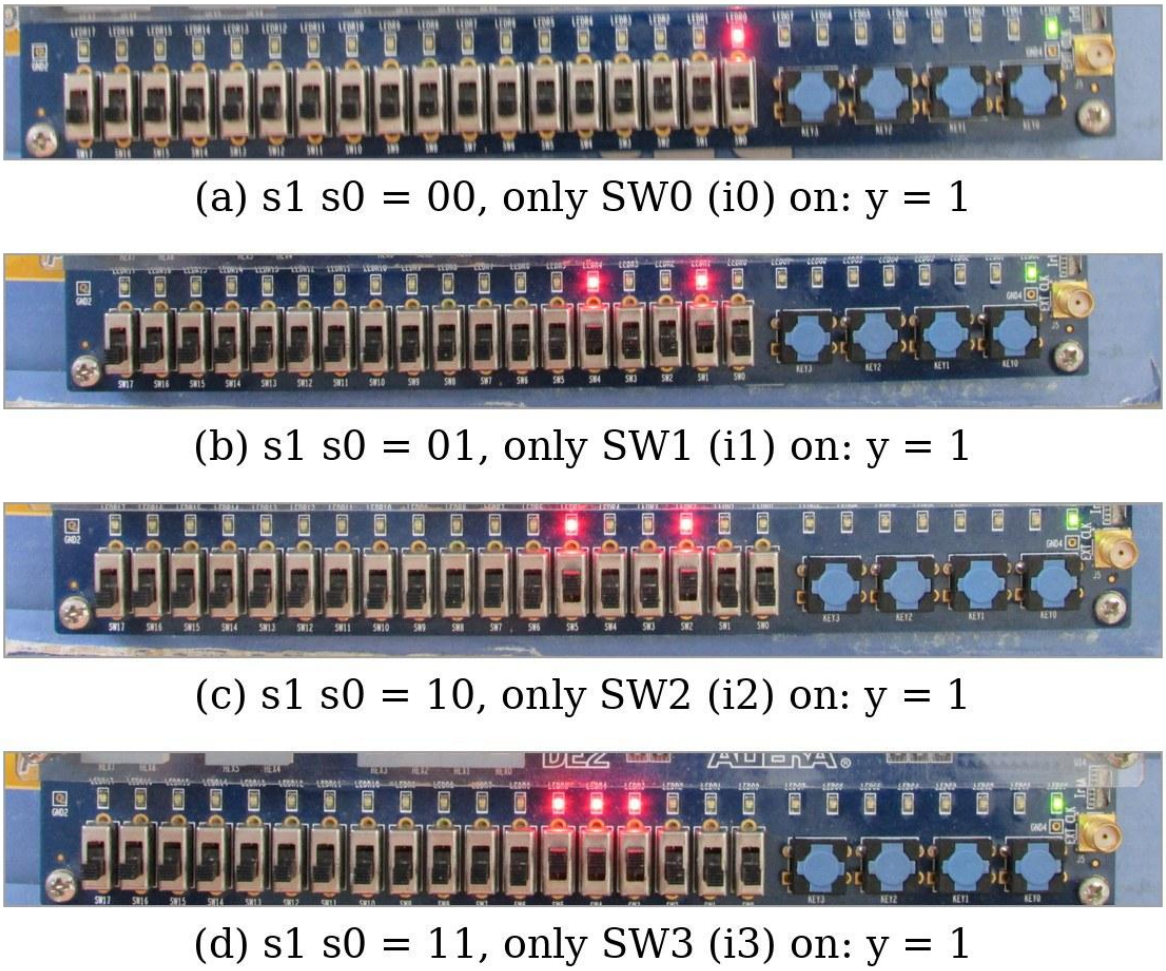
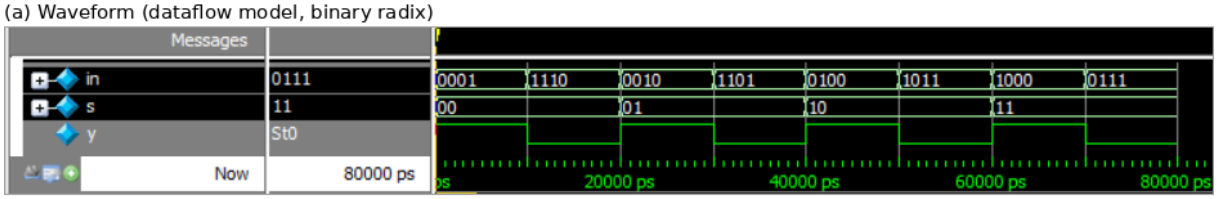


Figure 18: *FPGA implementation results of Experiment 6.*

Figure 19 shows the simulation results: (a) the waveform of the dataflow model (binary radix), in which, for each select value, first only the selected input is 1 and then only

the selected input is 0, so y always follows the selected input; (b)–(d) the transcripts of the three models, all reporting PASSED.

(a) Waveform (dataflow model, binary radix)



(b) Transcript – dataflow model

```
VSIM 32> run -all
# s1 s0 | i3 i2 i1 i0 | y
#  0  0 | 0  0  0  1 | 1
#  0  0 | 1  1  1  0 | 0
#  0  1 | 0  0  1  0 | 1
#  0  1 | 1  1  0  1 | 0
#  1  0 | 0  1  0  0 | 1
#  1  0 | 1  0  1  1 | 0
#  1  1 | 1  0  0  0 | 1
#  1  1 | 0  1  1  1 | 0
# RESULT: PASSED - all 8 cases correct
# Break in Module tb_mux41_df at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex6_DF/tb_mux41_df.v line 28
```

(c) Transcript – behavioral model (mux41beh_v2)

```
VSIM 35> run -all
# s1 s0 | i3 i2 i1 i0 | y
#  0  0 | 0  0  0  1 | 1
#  0  0 | 1  1  1  0 | 0
#  0  1 | 0  0  1  0 | 1
#  0  1 | 1  1  0  1 | 0
#  1  0 | 0  1  0  0 | 1
#  1  0 | 1  0  1  1 | 0
#  1  1 | 1  0  0  0 | 1
#  1  1 | 0  1  1  1 | 0
# RESULT: PASSED - all 8 cases correct
# Break in Module tb_mux41beh_v2 at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex6_BH/tb_mux41beh_v2.v line 28
```

(d) Transcript – structural model

```
VSIM 38> run -all
# s1 s0 | i3 i2 i1 i0 | y
#  0  0 | 0  0  0  1 | 1
#  0  0 | 1  1  1  0 | 0
#  0  1 | 0  0  1  0 | 1
#  0  1 | 1  1  0  1 | 0
#  1  0 | 0  1  0  0 | 1
#  1  0 | 1  0  1  1 | 0
#  1  1 | 1  0  0  0 | 1
#  1  1 | 0  1  1  1 | 0
# RESULT: PASSED - all 8 cases correct
# Break in Module tb_mux41_strutural at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex6_STRU/tb_mux41_strutural.v line 28
```

Figure 19: *Simulation results of Experiment 6.*

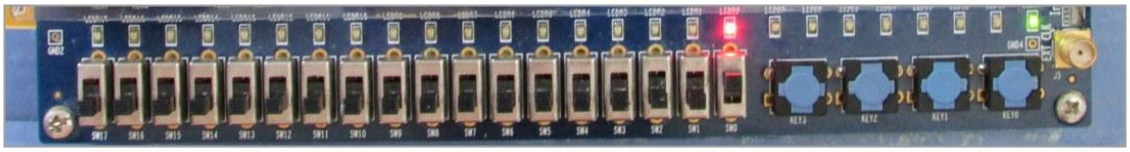
3.7 Experiment 7

Table 18 lists representative tests of the 16:1 MUX. As in Experiment 6, only the selected data input was set to 1 (LEDG0 on), and then it was cleared while other inputs were set to 1 (LEDG0 off).

Table 18: *Experiment 7 – expected outputs and observed results.*

S3 S2 S1 S0	Selected	Input set to 1	y (LEDG0)	STRU
0000	D0	SW0	1	✓
0101	D5	SW5	1	✓
1010	D10	SW10	1	✓
1101	D13	SW13	1	✓
1110	D14	KEY0 pressed	1	✓
1111	D15	KEY1 pressed	1	✓

Figure 20 shows the six tests of Table 18; the red LEDs mirror the switches and the green LEDG0 (right) shows the output y.



(a) S = 0000, SW0 on: y = 1



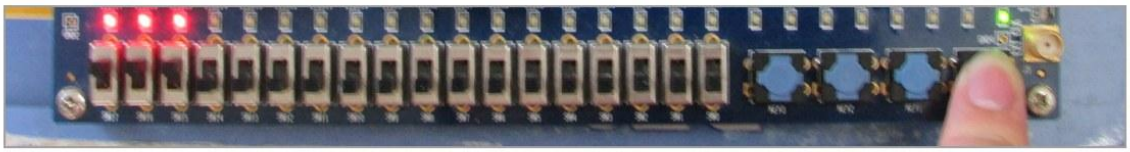
(b) S = 0101, SW5 on: y = 1



(c) S = 1010, SW10 on: y = 1



(d) S = 1101, SW13 on: y = 1



(e) S = 1110, KEY0 pressed: y = 1

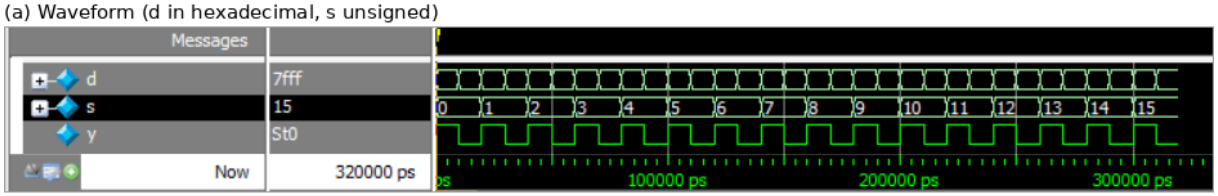


(f) S = 1111, KEY1 pressed: y = 1

Figure 20: *FPGA implementation results of Experiment 7.*

Figure 21 shows the simulation results: (a) the waveform (d in hexadecimal, s unsigned), in which, for each of the 16 select values, first only D[s] is 1 (y = 1) and then only D[s] is 0 (y = 0); (b) the transcript reporting PASSED for all 32 cases.

(a) Waveform (d in hexadecimal, s unsigned)



(b) Transcript

```
VSIM 42> run -all
# S3S2S1S0 | sel. | y (sel. = 1) | y (sel. = 0)
#   0000  | D0    |    1         |    0
#   0001  | D1    |    1         |    0
#   0010  | D2    |    1         |    0
#   0011  | D3    |    1         |    0
#   0100  | D4    |    1         |    0
#   0101  | D5    |    1         |    0
#   0110  | D6    |    1         |    0
#   0111  | D7    |    1         |    0
#   1000  | D8    |    1         |    0
#   1001  | D9    |    1         |    0
#   1010  | D10   |    1         |    0
#   1011  | D11   |    1         |    0
#   1100  | D12   |    1         |    0
#   1101  | D13   |    1         |    0
#   1110  | D14   |    1         |    0
#   1111  | D15   |    1         |    0
# RESULT: PASSED - all 32 cases correct
# Break in Module tb_mux161_structural at D:/Altera/90/quartus/Lab01/Lab1_Testbench/Ex7_STRU/tb_mux161_structural.v line 25
```

Figure 21: *Simulation results of Experiment 7.*

4 Discussion of Results

For all seven experiments, the outputs observed on the DE2 board agree with the truth tables derived in Section 1. The behavior of each circuit can be explained directly by its Boolean expression. In the half and full adders, the sum output follows the parity of the inputs (XOR) and the carry output is HIGH only when the arithmetic sum is at least 2; for example, A = B = 1 gives $1 + 1 = 10_2$, so S = 0 and Cout = 1. In the 4-bit adder, reading {LEDG4, LEDG3–LEDG0} as a binary number gives the decimal sum, and the test $1111_2 + 0000_2 + 1 = 10000_2$ confirms that the carry propagates correctly through all four full-adder stages. In the multiplexers, the output always follows the selected input and ignores the others, which was confirmed by setting all non-selected inputs to the opposite value.

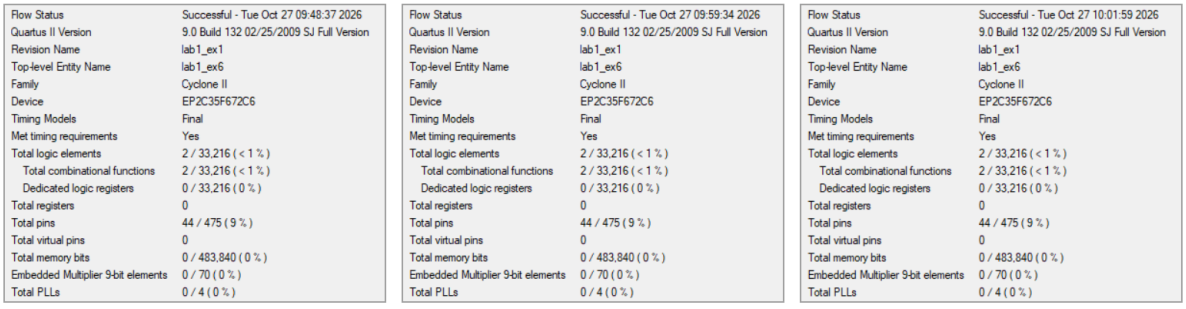
4.1 Comparison of the Modeling Styles

In every experiment the dataflow, behavioral and gate-level models produced identical results, as expected, because they describe the same Boolean functions. The styles differ in readability and in how close they are to the hardware:

- The **dataflow** model is the most compact, especially for arithmetic: the 4-bit adder is a single expression `A+B+Cin`, and the synthesis tool chooses the adder structure.
- The **behavioral** model describes the function as an algorithm (`if`, `case`, `for`), which is intuitive for multiplexers and truth-table descriptions, but the output signals must be declared as `reg` and every input combination must be covered to avoid unintended latches.
- The **gate-level** model gives full control of the circuit structure and maps one-to-one to the schematics, but it is the longest and most error-prone description, as shown by the errors found in the structural full adder.

After synthesis, Quartus II reduces all three descriptions to look-up-table logic in the FPGA, so the resource usage is almost independent of the modeling style. This is confirmed by the compilation reports of Experiment 6 (Figure 22): the dataflow, behavioral and structural 4:1 MUX models each use exactly 2 of 33,216 logic elements (< 1%) and 44 pins. Likewise, the seven logic gates of Experiment 1 use only 3 logic elements, all three 2:1 MUX models use only 1 logic element, and the 16:1 MUX built from five 4:1 MUX modules uses

only 10 logic elements. A small difference appears only in the 4-bit adder: the structural model uses 9 logic elements, whereas the behavioral model with the `for` loop uses 10. Both descriptions are functionally identical, but the synthesis tool starts from different netlists and its optimization does not always reach exactly the same minimum. Such differences are negligible for circuits of this size.



Flow Status	Successful - Tue Oct 27 09:48:37 2026
Quartus II Version	9.0 Build 132 02/25/2009 SJ Full Version
Revision Name	lab1_ex1
Top-level Entity Name	lab1_ex6
Family	Cyclone II
Device	EP2C35F672C6
Timing Models	Final
Met timing requirements	Yes
Total logic elements	2 / 33,216 (< 1 %)
Total combinational functions	2 / 33,216 (< 1 %)
Dedicated logic registers	0 / 33,216 (0 %)
Total registers	0
Total pins	44 / 475 (9 %)
Total virtual pins	0
Total memory bits	0 / 483,840 (0 %)
Embedded Multiplier 9-bit elements	0 / 70 (0 %)
Total PLLs	0 / 4 (0 %)

(a) Dataflow

Flow Status	Successful - Tue Oct 27 09:59:34 2026
Quartus II Version	9.0 Build 132 02/25/2009 SJ Full Version
Revision Name	lab1_ex1
Top-level Entity Name	lab1_ex6
Family	Cyclone II
Device	EP2C35F672C6
Timing Models	Final
Met timing requirements	Yes
Total logic elements	2 / 33,216 (< 1 %)
Total combinational functions	2 / 33,216 (< 1 %)
Dedicated logic registers	0 / 33,216 (0 %)
Total registers	0
Total pins	44 / 475 (9 %)
Total virtual pins	0
Total memory bits	0 / 483,840 (0 %)
Embedded Multiplier 9-bit elements	0 / 70 (0 %)
Total PLLs	0 / 4 (0 %)

(b) Behavioral

Flow Status	Successful - Tue Oct 27 10:01:59 2026
Quartus II Version	9.0 Build 132 02/25/2009 SJ Full Version
Revision Name	lab1_ex1
Top-level Entity Name	lab1_ex6
Family	Cyclone II
Device	EP2C35F672C6
Timing Models	Final
Met timing requirements	Yes
Total logic elements	2 / 33,216 (< 1 %)
Total combinational functions	2 / 33,216 (< 1 %)
Dedicated logic registers	0 / 33,216 (0 %)
Total registers	0
Total pins	44 / 475 (9 %)
Total virtual pins	0
Total memory bits	0 / 483,840 (0 %)
Embedded Multiplier 9-bit elements	0 / 70 (0 %)
Total PLLs	0 / 4 (0 %)

(c) Structural

Figure 22: *Flow Summary of the three 4:1 MUX models (Experiment 6).*

4.2 Errors Found in the Reference Code

Table 19 summarizes the errors found in the reference code of the lab manual and their corrections. Errors that only cause compilation failures are easy to detect, while logic errors such as the wrong `case` expression or the wrong OR-gate inputs compile without any error and can only be found by simulation or by testing on the board. This shows the importance of verifying every model against the truth table.

Table 19: *Errors found in the reference code and their corrections.*

Exp.	Module	Error	Effect	Correction
1	allgate_BH	`assign` inside `always`	Not synthesizable	Removed `assign`
1	lab1_ex1	Only 8 of 9 ports connected	XNOR output not displayed	Added LEDG[6]
3	full_adder_BH1	Missing `module` / `endmodule`	Compilation error	Keywords added
3	full_adder_BH1	`case (A | B | Cin)`	Wrong S and Cout for 4 of 8 inputs (acts as an OR gate)	`case ({A,B,Cin})`
3	full_adder (if-else)	Extra `endmodule`	Compilation error	Not used in this lab
3	full_adder_STRU	Port names `a, cin, Cum` vs. `A, Cin, Sum`	Compilation error	Port list corrected
3	full_adder_STRU	`or g5(Carry,x,y)`	Carry = A ⊕ B; e.g. A = B = 1 gives Carry = 0	`or g5(Carry,y,z)`
4	four_bit_adder_STRU	Uses the faulty full_adder_STRU	Wrong sums	Corrected full adder reused
6	mux41beh_v1	s[0] used as select MSB	Inputs i1 and i2 swapped	mux41beh_v2 used instead

4.3 Practical Issues and Sources of Error

Since the circuits are purely digital, there is no measurement error in the usual sense; the observed outputs either match the truth table or not. The following practical issues were encountered and must be considered as possible sources of wrong results:

- **Wrong pin assignment file.** The DE2-115 pin file refers to pins that do not exist on the Cyclone II package of the DE2 board; the board-specific file must be used.
- **Board not powered.** When the board was off, the USB-Blaster was detected but the JTAG chain could not be read ("JTAG chain broken"), so the FPGA could not be programmed.
- **Outdated programming file.** When a project was copied from a previous experiment, the old .sof file remained in the folder. Programming it before a successful recompilation loaded the previous circuit (in Experiment 3, SW3 seemed to have no effect). The design must be recompiled and the top-level entity checked before programming.
- **Switch numbering and port order.** SW0 is the rightmost switch, and the port order of the instantiated modules differs between models (e.g. the behavioral half adder lists its outputs first); a wrong order compiles without error but connects the wrong signals.
- **Limited I/O.** The 16:1 MUX requires 20 inputs, more than the 18 switches of the board, so two active-low push-buttons had to be used and inverted.

4.4 Conclusion

Seven basic combinational circuits (logic gates, half adder, full adder, 4-bit ripple carry adder, and 2:1, 4:1 and 16:1 multiplexers) were described in Verilog using the dataflow, behavioral and gate-level styles and were implemented on the DE2 FPGA board. All 19 models produced outputs identical to the theoretical truth tables, both in the ModelSim simulations and on the FPGA board. The experiments also showed that the three modeling styles are functionally equivalent and that careful verification is necessary, because several errors in the reference code compiled without warnings but produced wrong results.